

南開大學
Nankai University

南开大学

密码与网络空间安全学院

《计算机网络》实验报告

Lab1：Socket 编程

学　院	密码与网络空间安全学院
专　业	信息安全
学　号	2410826
姓　名	马伊汀
指导教师	徐敬东、张建忠

摘要

本次实验利用流式套接字（TCP）实现了一个支持多人在线聊天的应用程序。程序采用 C/S 架构，服务端基于 Winsock 多线程模型为每个客户端建立独立会话，客户端之间不直接通信，所有消息经服务端统一转发。设计并实现了 Ting/1.0 应用层文本协议，采用 4 字节大端长度前缀解决 TCP 字节流的粘包与半包问题，通过 UTF-8 编码支持中英文混排，利用服务端统一打时间戳与全局自增序号保证所有客户端看到相同的消息顺序，协议还包含心跳检测、优雅退出与序号连续性校验机制。经 200 条消息压力测试与 Wireshark 抓包验证，程序在应用层无消息丢失、无乱序，网络层 TCP 重传数为 0。

关键字：流式套接字　TCP　自定义应用层协议　多人聊天程序　Winsock　网络编程

目录

1 实验要求 **4**
2 GitHub 仓库 **4**
3 实验原理 **4**
3.1 分层结构与 TCP/IP 体系 . . . 4
3.2 协议与协议的三要素 . . . 4
3.3 应用进程的交互模式与进程标识 . . . 5
3.4 套接字与流式套接字 . . . 5
3.5 本实验使用的基本 Socket 函数 . . . 5
4 协议设计 **5**
4.1 总体设计 . . . 5
4.2 帧格式 . . . 6
4.3 消息类型 . . . 6
4.4 交互时序 . . . 7
4.5 退出机制与心跳 . . . 7
4.6 序号与丢包检测 . . . 8
5 程序设计 **8**
5.1 总体结构与模块划分 . . . 8
5.2 服务端主流程 . . . 9
5.3 服务端会话线程流程 . . . 10
5.4 客户端主流程 . . . 10
5.5 实验过程 . . . 11
6 各模块功能与核心代码 **12**
6.1 客户端（client/client.cpp） . . . 12
6.1.1 连接与登录 . . . 12
6.1.2 发送聊天消息 . . . 13
6.1.3 接收循环与消息分发 . . . 13
6.1.4 序号连续性校验 . . . 14
6.1.5 退出 . . . 14
6.2 服务端（server/server.cpp） . . . 15
6.2.1 监听与接入 . . . 15
6.2.2 登录校验 . . . 15
6.2.3 聊天处理与广播 . . . 16
6.2.4 心跳 . . . 17
6.2.5 停止服务 . . . 17
6.3 协议编解码（common/protocol.cpp） . . . 18
6.4 网络原语（common/net.cpp） . . . 19
6.5 丢包自测工具（tools/proto_test.cpp） . . . 20
7 程序界面展示及运行说明 **21**
7.1 启动服务端 . . . 21
7.2 客户端未连接状态 . . . 21
7.3 客户端连接成功 . . . 22
7.4 多人聊天 . . . 22
7.5 退出聊天室 . . . 24
7.6 丢包观察 . . . 25
7.6.1 应用层的自测工具 . . . 25
7.6.2 抓包验证 . . . 26
7.7 运行说明 . . . 27

8 实验过程中遇到的问题及分析 **27**
9 心得体会 **28**
附录 源代码目录结构 **29**

1 实验要求

本次实验要求利用流式套接字（TCP）实现一个多人在线聊天程序，具体要求如下：

- 使用流式 Socket，设计一个聊天协议，要求聊天信息带时间；请完整说明交互消息的语法、语义、时序等具体消息处理方式；
- 设计聊天程序，给出模块划分说明、模块功能和流程图；
- 利用 C 或 C++ 语言，使用基本 Socket 函数进行程序编写，不允许使用封装后的 Socket 类或架构；程序界面可用命令行方式，但需给出使用方法；
- 应用程序应支持英文和中文聊天；
- 采用多线程，支持多人聊天；
- 编写的程序应结构清晰，具有较好的可读性；
- 观察是否有数据包的丢失；
- 撰写实验报告，并将实验报告和源码提交至网站。

2 GitHub 仓库

本次实验的完整源码、可执行文件与文档已上传至 GitHub：

```
https://github.com/Myting24/Computer_Networks
```

3 实验原理

3.1 分层结构与 TCP/IP 体系

计算机网络是一个复杂的系统，它采用了**分层**的思想：把通信功能分解成若干层次，每一层只解决一部分问题，并且只通过接口向上一层提供服务，这样做的直接好处是，各层的实现可以独立演进，下层的传输介质从双绞线换成光纤，上层的应用不需要任何改动。

TCP/IP 体系结构把网络自上而下分为四层，分别是**应用层、传输层、互联层、网络接口层**。应用层为用户提供应用服务，数据单元称为报文；传输层为不同主机上的应用进程提供端到端传输，核心协议是 TCP 与 UDP；互联层负责主机到主机的数据包传输与路由选择；网络接口层负责把数据帧变成电信号发送出去。

每一层在发送数据时都会给上层数据加上本层的首部，接收时再逐层剥离，这一过程称为**封装与解封装**，TCP 负责把字节可靠、有序地送到对端，但“这些字节如何切分成一条条消息”是应用层自己的责任。

3.2 协议与协议的三要素

协议是对等实体之间共同遵守的约定和规则，一个完整的协议包含三个要素：**语法**、**语义**和**时序**。

语法定义“怎么做”，也就是报文或分组的格式、字段的数量与长度；语义定义“做什么”，即每个字段的具体含义；时序定义“何时做”，即何时发送何种报文、需要等待多久。在本次实验中，语法

对应“4 字节长度加负载、字段用制表符分隔”的帧格式，语义对应 HELLO、WELCOME、CHAT 等消息类型各自的含义，时序对应“连接后 10 秒内必须发 HELLO、服务端先回 WELCOME 再广播 JOIN”这类先后顺序。

3.3 应用进程的交互模式与进程标识

应用进程之间的交互主要有两种模式，**客户-服务器（C/S）模式**中，服务器进程处于守候状态，等待客户进程的请求并给出响应，典型应用有 DNS、WWW 和电子邮件等；**对等（P2P）模式**中各对等方功能相同、地位相等，最小化服务器的使用。在本次实验中，我们采用 C/S 模式：一个服务端进程守候在固定端口，多个客户端进程主动连接。

进程之间要发送和接收消息，必须有标识符，仅用 IP 地址无法区分同一主机上的多个进程，因此进程的地址标识是“**IP 地址 + 端口号**”：IP 地址标识哪一台主机，端口号标识该主机上的哪一个应用进程。服务端监听在 0.0.0.0:8888，客户端使用系统分配的临时端口，因此在服务端日志里看到的 127.0.0.1:58414 这类地址，前面的 IP 是客户端主机，后面的数字就是客户端进程临时占用的端口，这正是“进程用 IP 加端口标识”的直接体现。

3.4 套接字与流式套接字

TCP/IP 协议通常实现在操作系统内核中，应用程序不能直接操作网卡，需要一组编程界面来使用内核的网络功能，**套接字（socket）**就是操作系统为网络程序开发提供的典型编程界面。

按使用的传输层协议不同，套接字分为数据报套接字与流式套接字两类。前者使用 UDP，无连接、不可靠；后者使用 TCP，支持**面向连接、可靠的、有序的、全双工**的字节流传输，本次实验我们使用的就是流式套接字。

流式套接字的数据像水流一样连续，没有消息边界。多次 send 的数据可能被对端一次 recv 收到（粘包），一次 send 的数据也可能被拆成多次 recv 接收（半包）。因此“按固定长度收发”或“按换行分隔”都容易出错，必须由协议明确地划分消息边界。

3.5 本实验使用的基本 Socket 函数

本次实验只使用基本 Socket 函数，没有使用 CSocket 一类的封装类。服务端依次调用 WSAStartup 初始化 Winsock、socket 创建套接字（AF_INET、SOCK_STREAM）、bind 绑定地址与端口、listen 进入监听，然后在循环中调用 select 等待并调用 accept 取出连接；客户端则调用 socket 与 connect 发起连接。连接建立后，两端都用 send 与 recv 收发字节流，并用 select、setsockopt 控制等待时间，最后用 shutdown 与 closesocket 关闭连接。

其中 accept 需要特别说明：listen 的套接字只负责迎客，每来一个客户端，accept 会返回一个新的套接字专门服务该客户端，服务端之所以能同时服务多人，正是因为可以为每个新套接字单独开一个线程。另外，网络传输使用大端字节序（网络字节序），而 x86 主机是小端，因此协议中的长度字段需要按大端手工组装与解析。

4 协议设计

4.1 总体设计

协议命名为 **Ting/1.0**，是一个运行在 TCP 之上的自定义应用层文本协议，使用端口 8888。协议的设计目标与对应手段可以概括为以下五点：

- 为了解决粘包与半包，采用 4 字节大端长度前缀来明确划分消息边界；
- 为了支持中文、英文、标点等任意字符，负载统一使用 UTF-8 编码且长度按字节计算；
- 为了区分每条消息的用途，负载的第一个字段固定为消息类型；
- 为了让一条消息在界面上占一行，字段之间用制表符分隔，正文中的换行在服务端被替换为空格；
- 为了让所有客户端看到相同的顺序与时间，由服务端统一打时间戳并分配全局递增序号。

程序分为服务端与客户端两个可执行文件，客户端之间不直接通信，所有消息都经过服务端转发，服务端可以统一决定顺序、时间与序号，保证所有客户端看到的内容完全一致。

4.2 帧格式

每条消息（协议中称为一帧）由两部分组成：前面是 **4 个字节的无符号整数**，以**大端**方式表示负载长度 L；后面是 L **个字节的负载**，内容是 UTF-8 文本，负载中的第一个字段是消息类型，其后各字段用制表符分隔。

由于字段之间用制表符分隔，字段内部若出现反斜杠、制表符、回车或换行，需要分别转义为 \\、\t、\r、\n，避免一条消息被错拆成多段，转义与反转义都在协议模块中集中完成。

以客户端登录、昵称为“张三”为例，其帧共 20 个字节，内容如下：

```
00 00 00 10 48 45 4C 4C 4F 09 31 2E 30 09  E5 BC A0 E4 B8 89
```

前 4 个字节 00 00 00 10 表示负载长度为 16；接着是类型 HELLO（ASCII 码 48 45 4C 4C 4F）；然后是制表符 09；再是版本号 1.0；再一个制表符；最后 6 个字节是“张三”的 UTF-8 编码。可以看到，长度字段统计的是**字节数**而不是字符数，这正是中英文混排不会出错的原因。

采用长度前缀而不是换行分隔，是因为 TCP 是字节流、没有消息边界，接收方先读固定的 4 个字节得到长度 L，再精确读满 L 个字节，得到的就是完整的一帧，既不会把两条消息粘在一起，也不会只读到半条，同时，正文中允许出现任意字符，不影响分帧。

4.3 消息类型

协议共定义 11 种消息类型，每种类型的字段顺序与其在负载中的顺序一致，如表 4.1 所示。

类型	方向	字段	语义
HELLO	C→S	版本、昵称	登录请求，必须是连接后的第一帧
WELCOME	S→C	版本、服务端标识、时间、编号、昵称、在线人数	登录成功，告知分配的编号与最终昵称
JOIN	S→C	序号、时间、编号、昵称、在线人数	有人加入，广播给所有人（含本人）
LEAVE	S→C	序号、时间、编号、昵称、原因、在线人数	有人离开，原因为主动退出、连接中断、心跳超时或服务器关闭
CHAT	C→S	本地序号、正文	客户端发送聊天正文
CHAT	S→C	全局序号、时间、编号、昵称、正文	服务端广播聊天（含发送者本人）
USERLIST	S→C	人数、(编号、昵称) × 人数	在线用户全量快照
SYSTEM	S→C	序号、时间、公告	系统公告，如服务器关闭
ERROR	S→C	错误码、描述	出错应答
PING	S→C	服务端时间	心跳请求
PONG	C→S	原样回带的 PING 时间	心跳应答
BYE	C→S	无	客户端主动退出

表 4.1: Ting/1.0 协议消息类型一览

其中 CHAT 有两个方向、字段含义不同，客户端上行时第一个字段是**本地序号**，客户端自己的发送计数，仅用于服务端日志对账；服务端下行时第一个字段是**全局序号**，服务端为广播事件统一编号，用于排序与丢包检测。

设计上有三处我们在这里单独说明一下：第一，服务端把聊天消息**也发给发送者本人**，这样所有人（包括发送者）看到的时间、序号和内容完全一致；第二，成员变化统一用 USERLIST **全量快照**刷新，不容易出错；第三，WELCOME 会返回**最终昵称**，因为服务端对重名昵称会自动追加 #2，客户端必须以服务端返回的为准。

4.4 交互时序

1、登录与加入。

客户端通过三次握手建立 TCP 连接后，必须在 10 秒内发送 HELLO，其中带有协议版本与期望昵称。服务端校验版本、昵称长度与非法字符，通过后分配一个客户端编号，若昵称重复则改为“张三 #2”，随后依次向该客户端发送 WELCOME（告知编号与最终昵称）、JOIN（宣布自己加入）和 USERLIST（全量在线名单），同时向其他所有客户端广播 JOIN 与新的 USERLIST。客户端收到 WELCOME 后状态变为“在线”，输入框才允许发送。

2、聊天。

客户端把正文转成 UTF-8，连同自己的本地序号组成 CHAT 帧发出，服务端收到后先校验长度，单条正文不超过 4000 字节，再把正文中的换行替换为空格，然后在广播锁保护下追加全局序号与时间戳，连同发送者编号和昵称一起广播给所有在线客户端（包含发送者）。由于所有客户端收到的是同一份字节，显示内容、时间和序号必然一致。

3、退出。

客户端点击“断开”时先发送 BYE，再关闭连接，服务端读到 BYE 后把离开原因记为“主动退出”，把该会话移出在线表，然后广播 LEAVE（带原因与剩余人数）和新的 USERLIST，最后关闭套接字。

4、异常。

如果客户端在 10 秒内没有发送 HELLO，服务端直接关闭连接；如果首帧不是 HELLO、版本不匹配或昵称非法，服务端回 ERROR 后关闭；如果客户端进程被强制结束或网络中断，服务端只能看到 recv 返回错误，原因记为“连接中断”；如果长时间没有任何数据往来，则由心跳机制判定超时，原因记为“心跳超时”；服务端主动停止时，先广播 SYSTEM 通知所有人，再统一断开，原因记为“服务器关闭”。

4.5 退出机制与心跳

我们在协议中定义了一条 BYE 消息：客户端在关闭连接之前先发送它，服务端读到 BYE 后把离开原因标记为“主动退出”再广播 LEAVE。由于 TCP 会先把已经写入发送缓冲区的数据送达、再发送 FIN，这一帧不会丢失。

但客户端被强制结束或网络中断时，可能根本来不及发送 BYE，为此服务端每 **15 秒**向所有在线客户端发送一次 PING，客户端收到后立即回 PONG。如果某个连接在 **75 秒**内没有收到任何数据（既没有聊天，也没有 PONG），就判定该客户端已经掉线，把它移出在线名单并广播 LEAVE（原因“心跳超时”）。这样既能在正常退出时准确区分原因，也能在异常掉线时及时清理名单，不会长期保留“僵尸用户”。

4.6 序号与丢包检测

服务端维护一个全局自增序号，每产生一次广播事件（JOIN、LEAVE、CHAT、SYSTEM）就加一，并写入该帧的第一个字段。由于“取序号”和“广播”这两个动作是在同一把互斥锁内完成的，广播被严格串行化，因此所有客户端收到的序号一定是连续递增的。

客户端记录“期望收到的下一个序号”，每收到一条带序号的事件就与之比对：如果相等，说明顺序正常，把期望值加一并继续；如果不相等，说明中间有事件没有收到，就在聊天区打印一条警告，同时把状态栏里的“序号跳跃”计数加一，便于直观地看到是否发生过漏消息。

其中，USERLIST、WELCOME、ERROR、PING 属于状态同步或控制类消息，不占用序号，也不参与连续性判定，这样划分的原因是：前者是“全量快照”，本身具备自愈能力；后者是控制流，与聊天事件的先后顺序无关。

最后需要说明的是，TCP 本身是可靠传输，网络层丢掉的包会被自动重传，应用层通常看不到真正的丢包，因此本程序验证的其实是应用层有没有漏消息、乱序，例如分帧写错导致半条消息被吞掉、广播时漏发给某个客户端、解析失败被丢弃等。

5 程序设计

5.1 总体结构与模块划分

程序按职责分为四层，自下而上依次是网络层、协议层、业务层和界面层，各层的职责与对应的源文件如图 5.1 所示。

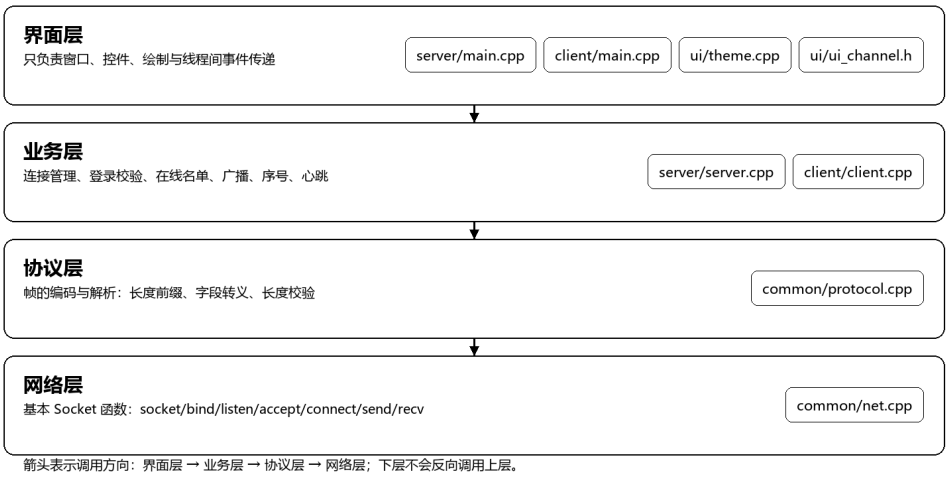


图 5.1: 程序模块划分

界面层（server/main.cpp、client/main.cpp、ui/theme.cpp、ui/ui_channel.h）只负责窗口、控件、绘制、DPI 适配与线程间事件传递，不包含任何网络逻辑，界面层位于最上方，它是用户唯一直接接触的部分，所以我们把界面单独隔离出来，后面调整配色、布局都不会影响通信逻辑。

业务层（server/server.cpp、client/client.cpp）是程序的核心，负责连接管理、登录校验、在线名单维护、消息广播、序号分配与心跳检测，服务端与客户端各自实现一套业务逻辑，但都建立在同一套协议与网络原语之上。

协议层（common/protocol.cpp）负责帧的编码与解析，包括长度前缀的读写、字段的转义与反转义、长度上限校验，所有涉及字节格式的细节都集中在这一个模块里，业务层只和“类型 + 字段”的逻辑帧打交道，看不到字节。

网络层（common/net.cpp）只做基本 Socket 函数本该做的事：建立连接、收发字节、设置超时、把错误码翻译成可读文字，它没有把套接字封装成对象，所有函数都直接接收 SOCKET 句柄。

还有两个公共模块:common/text.cpp 负责 UTF-8 与 UTF-16 的转换和时间格式化,common/logger.cpp 负责把日志写入 UTF-8 文件，它们的层次最低，任何一层都可以调用。

模块之间的调用方向是单向的：界面层调用业务层，业务层调用协议层与网络层，下层不会反向调用上层。界面层与业务层之间通过回调和线程安全的事件队列通信，因此网络线程永远不需要直接操作窗口。

5.2 服务端主流程

服务端的进程级流程如图 5.2 所示：初始化 Winsock 后创建套接字、绑定端口并进入监听，随后在循环中用 select 等待新连接；每来一个连接就 accept 出一个新套接字，并为它创建一个独立的会话线程，主线程立刻回到 select 继续等待。停止服务时，先广播系统公告，再依次关闭监听套接字、关闭所有会话、等待线程退出，最后清理 Winsock。

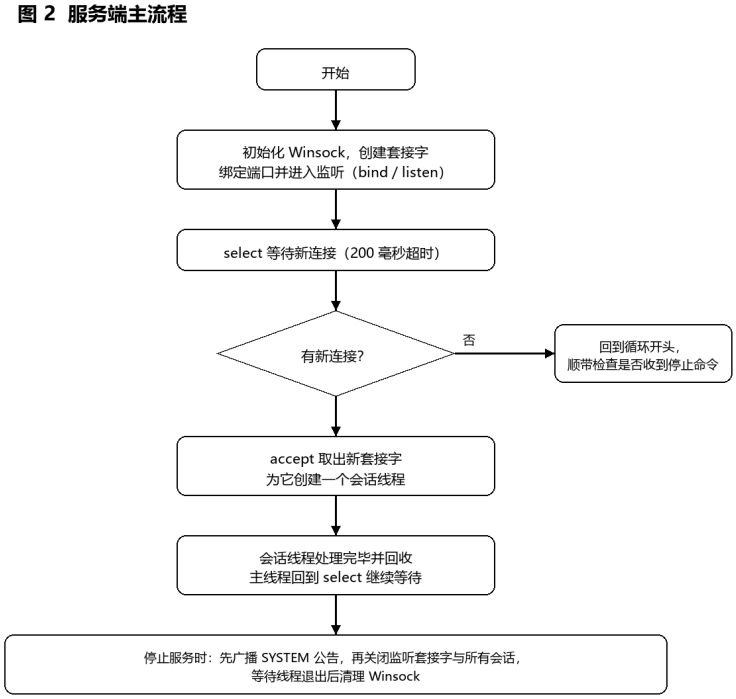


图 5.2: 服务端主流程

这里用 select 轮询是为了让“停止服务”能够被及时响应，因为直接阻塞时只能靠关闭套接字去唤醒 accept，行为不够可控。除此之外，每来一个连接就开一个线程是我们程序能同时服务很多人的关键，这样如果某个客户端发送很慢或长时间不说话，只会阻塞它自己的会话线程，不会影响其他人。

5.3 服务端会话线程流程

每个会话线程的生命周期如图 5.3 所示，可以分成三段：登录校验、消息循环和收尾清理。线程启动后先等待 HELLO（10 秒超时），校验版本与昵称合法后分配编号、加入在线名单，并发送 WELCOME、广播 JOIN 与 USERLIST；随后进入消息循环，按类型分发处理；循环结束后记录离开原因、移出在线名单、广播 LEAVE 与新的 USERLIST，最后关闭套接字。

图 3 服务端会话线程流程

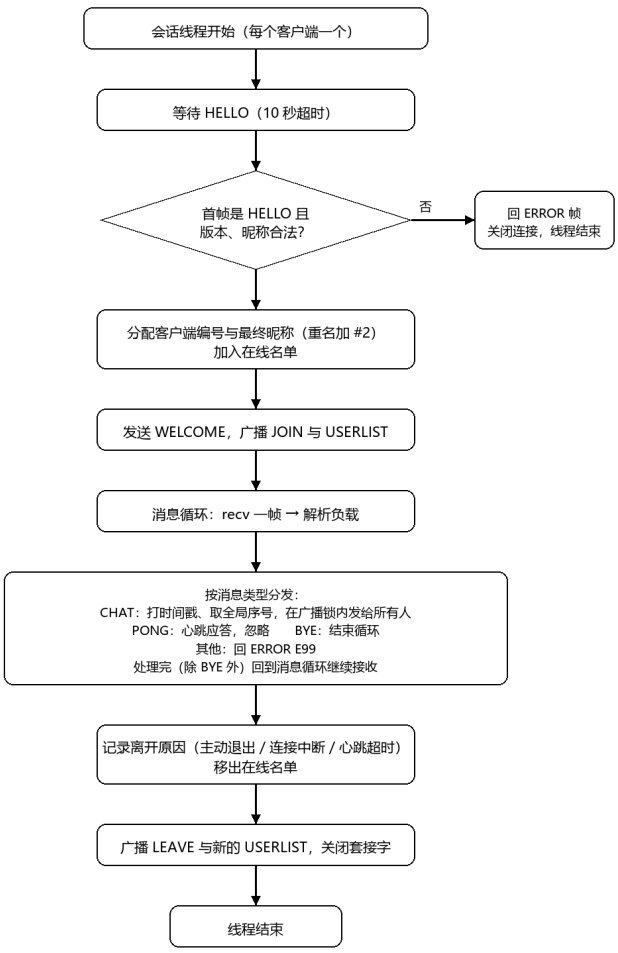


图 5.3: 服务端会话线程流程

首先，**登录校验放在消息循环之前**，因为它是唯一必须“按顺序来”的环节，第一帧不是 HELLO 就直接断开，避免不合规的连接占用资源。其次，**无论是主动退出、连接中断还是心跳超时，最后都走同一段收尾逻辑**，这样在线名单、广播内容和日志格式保持一致，不会因为退出方式不同而出现差异。

5.4 客户端主流程

客户端的主流程如图 5.4 所示：点击“连接”后创建接收线程，先 connect（最多等 5 秒），成功后发送 HELLO，收到 WELCOME 才把状态切换为“在线”并进入接收循环；接收循环按消息类型分发处理；用户点击“断开”或连接中断时，发送 BYE、shutdown 唤醒接收线程、join 线程，最后关闭套接字并回到“未连接”状态。

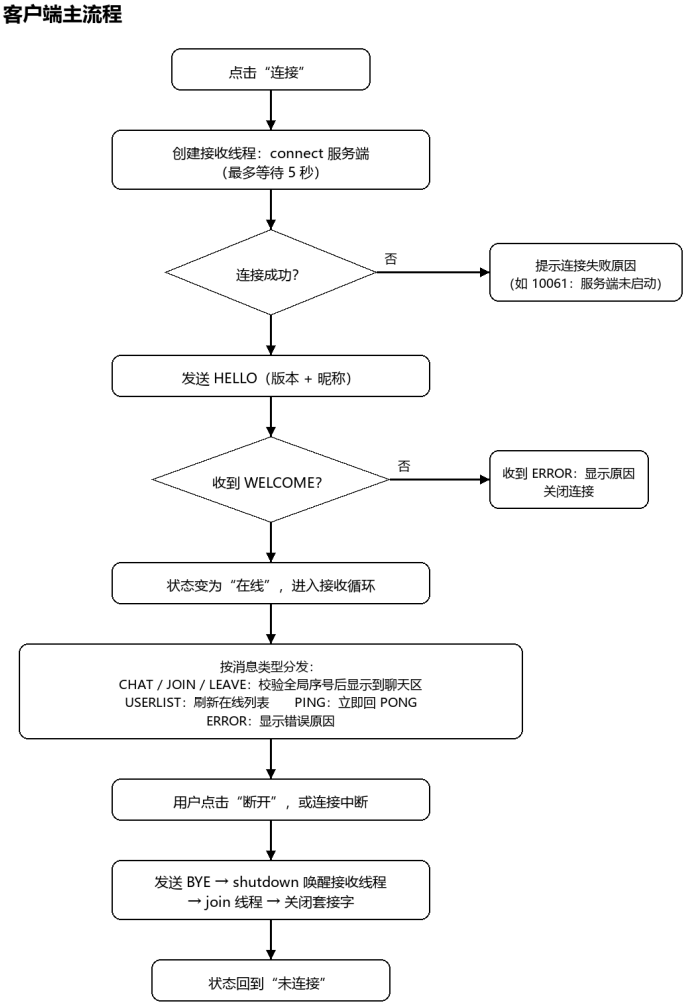


图 5.4: 客户端主流程

客户端有两个线程，我们必须保证界面线程不被网络阻塞，因此所有网络操作都在接收线程里完成，界面只负责显示和把用户输入交给发送函数；两个线程之间通过状态机（未连接、连接中、登录中、在线、关闭中）协调，任何时刻按钮的可用状态都由当前状态决定。

5.5 实验过程

整个实验大致按以下顺序推进，每一步都在前一步能跑通的基础上再增加功能。

首先是**确定协议**。在写任何代码之前，先明确帧格式（4 字节长度前缀加 UTF-8 负载）、消息类型（11 种）和交互时序（先 HELLO 再 WELCOME 再 JOIN），并对照“语法、语义、时序”三要素逐条检查有没有遗漏的字段。

其次是**实现网络层与协议层**。先写 common/net.cpp，把 socket、bind、listen、accept、connect、send、recv 这些调用封装成带错误处理与循环收发的函数，并单独测试“发一条消息能否完整收到”；再写 common/protocol.cpp，实现组帧与拆帧，用固定的十六进制数据验证编码结果是否正确。

第三步是**服务端**：先实现“监听 + 接受连接 + 回显”的最小闭环，确认用 telnet 或自测工具能连上并收到数据；再加入登录校验与在线名单；再实现广播与全局序号；最后加心跳线程，每加一个功能都用日志观察行为。

第四步是**客户端**：先实现“连接 + 登录 + 接收”的最小闭环，能收到服务端的欢迎语；再加入发送功能与序号校验；最后处理退出流程（先发 BYE 再关闭）。

第五步是**界面**。因为业务逻辑已经可以独立运行，界面只需要调用客户端的接口并把事件显示出来。

最后是**测试与文档**：用自测工具连续发送 200 条消息验证不丢不乱，用 Wireshark 抓包确认协议在网络上就是设计的样子，再整理设计文档、使用说明与实验报告。

6 各模块功能与核心代码

本章逐个模块分析核心代码，按“客户端、服务端、协议编解码、网络原语、丢包自测工具”的顺序展开。

6.1 客户端（client/client.cpp）

客户端是用户直接接触的一端，它的职责包括：维护连接状态机、完成登录握手、把用户输入编码成协议报文、持续接收并解析服务端广播、校验序号连续性，以及在退出时优雅地关闭连接。

6.1.1 连接与登录

点击“连接”后，界面线程调用 BeginConnect，由它创建一个工作线程执行 WorkerMain，真正的网络操作全部放在这个线程里，避免阻塞界面。WorkerMain 依次做三件事：连接、发 HELLO、进入接收循环。

```
void ChatClient::WorkerMain(std::wstring host, uint16_t port, std::wstring nick) {
    EmitState(State::Connecting,
            L"正在连接 " + host + L":" + std::to_wstring(port) + L" …");

    std::string error;
    SOCKET sock = net::Connect(text::WideToUtf8(host), port, 5000, &error);
    if (sock == INVALID_SOCKET) {
        EmitLine(L"【错误】连接服务器失败: " + text::Utf8ToWide(error));
        m_state.store(State::Idle);
        EmitState(State::Idle, L"未连接");
        return;
    }
    {
        std::lock_guard<std::mutex> lock(m_ioMutex);
        m_sock = sock;
    }
    net::SetNoDelay(sock);
    net::SetSendTimeout(sock, 3000);

    EmitState(State::LoggingIn, L"TCP 连接已建立，正在发送 HELLO 登录 …");
    std::wstring sendError;
    if (!SendFrame(scp::m::kHello, {scp::kVersion, text::WideToUtf8(nick)}, &sendError)) {
        /* 发送失败则关闭连接并回到未连接状态 */
        return;
    }
    RecvLoop(sock);   // 阻塞在这里，直到连接结束
}
```

第一，**状态是显式推进的**：从“正在连接”到“TCP 连接已建立，正在发送 HELLO”再到“在线”，每一步都通过 EmitState 通知界面，用户能看清程序在做什么。第二，**连接失败要给出可操作**

的原因：net::Connect 返回的错误里若含有 10061，说明目标端口没有服务端在监听，程序会额外提示“请先打开 Server.exe”。第三，**先发 HELLO 再进接收循环**：登录是所有后续消息的前提，必须先完成握手，再进入统一的收包处理。

6.1.2 发送聊天消息

发送环节的核心是把界面上的宽字符串转成 UTF-8，并附上本地序号。

```
bool ChatClient::SendChat(const std::wstring& text, std::wstring* err) {
    if (m_state.load() != State::Online) {
        if (err != nullptr) *err = L"尚未连接服务器";
        return false;
    }
    const std::wstring content = text::Trim(text);
    const std::string utf8 = text::WideToUtf8(content);
    if (utf8.size() > scp::kMaxChatBytes) {
        if (err != nullptr) *err = L"消息过长（单条最多 4000 字节，约 1300 个汉字）";
        return false;
    }
    ++m_localSeq;
    return SendFrame(scp::m::kChat, {std::to_string(m_localSeq), utf8}, err);
}
```

第一，**先检查状态再发送**：未连接时直接返回错误，而不是让帧发到一个已经关闭的套接字上。第二，**按字节判断超长**：kMaxChatBytes 限制的是 UTF-8 字节数，而不是字符数，因为网络上传输和分帧都以字节为单位。第三，**本地序号只用于日志**：它是“我这个客户端发出的第几条消息”，服务端只把它写进日志便于对账，真正决定顺序的是服务端分配的全局序号。

6.1.3 接收循环与消息分发

登录成功后，工作线程进入 RecvLoop 循环收帧。

```
void ChatClient::RecvLoop(SOCKET sock) {
    while (m_state.load() != State::Closing) {
        std::string payload;
        std::string error;
        if (!net::RecvFrame(sock, payload, &error)) {
            if (error.find("10060") != std::string::npos ||
                error.find("超时") != std::string::npos) {
                EmitLine(L"【提示】等待服务器数据超时，连接已断开。");
            } else {
                EmitLine(L"【提示】与服务器的连接已断开：" + text::Utf8ToWide(error));
            }
            return;
        }
        ++m_rxFrames;

        scp::Frame frame;
        if (!scp::ParsePayload(payload, frame, &error)) {
            EmitLine(L"【提示】收到无法解析的数据帧，已忽略。");
            continue;
        }
        if (!Dispatch(frame)) {
            return;
        }
    }
```

```
}
```

接收循环有三条原则：第一，**循环条件看状态**：一旦另一线程把状态置为 Closing（用户点了断开），循环就会退出，不需要强行终止线程。第二，**解析失败不影响运行**：单帧解析失败只打印提示并继续，因为一帧畸形数据不应该让整条连接断开。第三，**退出只走一条路径**：Dispatch 返回 false 表示“该结束了”，此时统一 return 回 WorkerMain 收尾，不会出现多处各自关闭套接字的情况。

分发函数按类型把消息翻译成界面上的文字。以聊天消息为例：

```
if (frame.type == scp::m::kChat) {
    CheckSequence(frame.fields.size() > 0 ? frame.fields[0] : std::string());
    EmitLine(text::Format(L"[%s] %s: %s", FieldTime(frame, 1).c_str(),
                      FieldText(frame, 3).c_str(), FieldText(frame, 4).c_str()));
    return true;
}
```

CheckSequence 先校验全局序号，再格式化输出，其中 FieldTime 取服务端时间，FieldText 取昵称与正文，这里显示的时间**不是本地时钟**，而是服务端写入帧里的时间，这样两个人看到的同一条消息时间相同。

6.1.4 序号连续性校验

```
void ChatClient::CheckSequence(const std::string& seqText) {
    unsigned long long seq = 0;
    try { seq = std::stoull(seqText); } catch (...) { return; }

    if (m_expectedSeq != 0 && seq != m_expectedSeq) {
        ++m_gaps;
        EmitLine(text::Format(L"【警告】消息序号不连续：期望 %llu，实际收到 %llu",
                          m_expectedSeq, seq));
    }
    m_expectedSeq = seq + 1;
    ++m_received;
}
```

这段代码实现了“丢包观察”功能，客户端记住“下一个应该收到的序号”，一旦实际收到的序号对不上，就说明中间有事件没有到达，此时既在聊天区打印警告，也把计数加一，供状态栏显示。判断 m_expectedSeq != 0 是因为第一次收到事件时还没有期望值，不能算作跳号；之所以用 try/catch 包住 stoull，是因为序号字段理论上可能被畸形数据污染，转换失败时直接忽略即可，不应该让异常穿过线程边界。

6.1.5 退出

```
void ChatClient::Disconnect() {
    if (m_state.load() == State::Online) {
        SendFrame(scp::m::kBye, {}, nullptr); // [1] 先发 BYE
    }
    m_state.store(State::Closing);

    {
        std::lock_guard<std::mutex> lock(m_ioMutex);
        if (m_sock != INVALID_SOCKET) {
            ::shutdown(m_sock, SD_BOTH);  // [2] 唤醒阻塞中的 recv
```

```
        }
    }
    if (m_thread.joinable()) {
        m_thread.join();                    // [3] 等接收线程退出
    }
    m_state.store(State::Idle);
}
```

退出顺序不能颠倒：第一步发 BYE，让服务端把这次离开记为“主动退出”并广播给其他人；第二步 shutdown，因为接收线程此时正阻塞在 recv 上，只有半关闭套接字才能让它立刻返回；第三步 join，等接收线程真正结束后再返回，否则界面可能在线程还在使用套接字时就把对象销毁了。这三步的顺序，是“先通知、再唤醒、后等待”的标准做法。

6.2 服务端（server/server.cpp）

服务端的职责是：监听端口、为每个客户端建立会话、校验登录、维护在线名单、把消息广播给所有人、维护全局序号、通过心跳发现掉线的客户端，并在退出时有序清理，它所有状态都是**多线程共享**的。

6.2.1 监听与接入

```
while (m_running.load()) {
    fd_set readSet;
    FD_ZERO(&readSet);
    FD_SET(m_listen, &readSet);
    timeval timeout{0, 200000};                // 200ms
    const int ready = ::select(0, &readSet, nullptr, nullptr, &timeout);
    if (!m_running.load()) break;
    if (ready <= 0) continue;

    sockaddr_in peer{};
    int peerLength = static_cast<int>(sizeof(peer));
    const SOCKET client = ::accept(m_listen, reinterpret_cast<sockaddr*>(&peer), &peerLength);
    if (client == INVALID_SOCKET) continue;

    // 为新连接创建会话对象，并开一个独立线程
    std::thread([this, session]() { SessionLoop(session); }).detach();
}
```

这里我们用 select 做 200 毫秒的轮询，线程每 200 毫秒检查一次 m_running，停止时能自然退出。accept 返回的**新套接字**交给一个独立的会话线程处理，主循环立刻回到 select 等待下一个连接，这样就能实现“每来一个人就开一个线程”。

6.2.2 登录校验

会话线程启动后，第一阶段是等待并校验 HELLO。

```
net::SetRecvTimeout(session->sock, kHelloTimeoutMs); // 10 秒
if (!net::RecvFrame(session->sock, payload, &error)) {
    Log(text::Format(L"[断开] %s 未完成登录：%s", peerText.c_str(),
                text::Utf8ToWide(error).c_str()));
    ::closesocket(session->sock);
    return;
}
```

```
++m_statFramesIn;

scp::Frame hello;
if (!scp::ParsePayload(payload, hello, &error) || hello.type != scp::m::kHello) {
    SendFrameTo(session, scp::m::kError, {"E01", "首个消息必须是 HELLO"});
    ::closesocket(session->sock);
    return;
}
if (hello.fields.size() < 2) {
    SendFrameTo(session, scp::m::kError, {"E02", "HELLO 字段不足"});
    ::closesocket(session->sock);
    return;
}
if (hello.fields[0] != scp::kVersion) {
    SendFrameTo(session, scp::m::kError,
                {"E03", "协议版本不匹配，服务端要求 " + std::string(scp::kVersion)});
    ::closesocket(session->sock);
    return;
}
```

登录阶段必须按顺序来：**第一帧是 HELLO**，我们设置了 10 秒的接收超时，防止有人连上来却不说话，长期占用一个线程。校验分三层依次进行：先看能不能收到帧，再看类型是不是 HELLO，最后看字段数量与版本号是否匹配。每一层失败都返回一个带错误码的 ERROR 帧再关闭连接，这样客户端能明确知道问题出在哪里（E01 类型错、E02 字段缺失、E03 版本不匹配）。

校验通过后，服务端为会话分配编号、生成最终昵称（重名时追加 #2），把它加入在线表，再依次发送 WELCOME、JOIN 和 USERLIST，同时向其他人广播新的成员变化，至此会话才真正进入“在线”状态。

6.2.3 聊天处理与广播

```
const std::string localSeq = frame.fields[0];
const std::string body = FlattenNewlines(contentUtf8);

{
    std::lock_guard<std::mutex> broadcastLock(m_broadcastMutex);
    std::vector<std::string> fields;
    NextEventHeader(fields);                // 全局序号 + 时间
    fields.push_back(std::to_string(session->id));
    fields.push_back(session->nick);
    fields.push_back(body);
    BroadcastFrameLocked(scp::m::kChat, fields);
}
++m_statMessages;
```

广播函数本身是这样实现的：

```
void ChatServer::BroadcastFrameLocked(const std::string& type,
                                      const std::vector<std::string>& fields,
                                      const SessionPtr& except) {
    scp::Frame frame;
    frame.type = type;
    frame.fields = fields;
    std::string wire;
    std::string error;
    if (!scp::BuildWire(frame, wire, &error)) {
```

```
        Log(L"[错误] 广播帧编码失败: " + text::Utf8ToWide(error));
        return;
    }
    const auto targets = Snapshot(true);
    for (const auto& target : targets) {
        if (target == except || target->dead.load()) continue;
        SendWire(target, wire);
    }
}
```

第一，**先编码一次，再发给所有人**：同一条广播在所有客户端必须逐字节相同，否则可能出现不一致，例如时间戳被重新取一次。第二，**取在线表快照再遍历**：如果直接持有在线表的锁去逐个发送，某个客户端发送阻塞时会把整张表锁住，其他线程就会被卡住，先拷贝一份快照再发送，锁的持有时间就缩短到可以忽略。每个会话各自持有一把发送锁，保证同一连接上的多帧不会交叉。

FlattenNewlines 把正文中的换行替换为空格，这样一条消息在界面上永远只占一行，避免用户发送多行文本把聊天区撑乱。

6.2.4 心跳

```
void ChatServer::HeartbeatLoop() {
    while (m_running.load()) {
        for (int i = 0; i < kHeartbeatSeconds * 10 && m_running.load(); ++i) {
            ::Sleep(100);                   // 分片睡眠，便于及时响应停止
        }
        if (!m_running.load()) break;

        std::lock_guard<std::mutex> broadcastLock(m_broadcastMutex);
        const std::string now = text::WideToUtf8(text::NowDateTime());
        std::vector<std::string> fields{now};
        BroadcastFrameLocked(scp::m::kPing, fields);
        const int online = static_cast<int>(Snapshot(true).size());
        Log(text::Format(L"[心跳] 已向 %d 位在线用户发送 PING", online));
    }
}
```

这里用“循环 150 次、每次睡 100 毫秒”来代替一次性 Sleep(15000)，是为了让服务端在停止时能在一百毫秒内响应。心跳线程发送 PING 时同样要拿广播锁，因为 PING 也是广播，必须和聊天广播串行，否则可能插在某个客户端的两次广播之间，破坏客户端对“序号连续”的判断。客户端收到 PING 后立即回 PONG，服务端把 PONG 视为“人还在”的证据；如果 75 秒内什么数据都没收到，会话线程的接收就会超时，从而按“心跳超时”清理这个客户端。

6.2.5 停止服务

```
void ChatServer::Stop() {
    if (!m_running.exchange(false)) { /* 未在运行 */ return; }

    // (1) 先通知在线客户端
    {
        std::lock_guard<std::mutex> broadcastLock(m_broadcastMutex);
        std::vector<std::string> fields;
        NextEventHeader(fields);
        fields.push_back(text::WideToUtf8(L"服务器正在关闭，连接即将断开。"));
        BroadcastFrameLocked(scp::m::kSystem, fields);
```

```
    }

    // (2) 等监听线程退出，再关闭监听套接字
    if (m_acceptThread.joinable()) m_acceptThread.join();
    SOCKET listenSocket = m_listen;
    m_listen = INVALID_SOCKET;
    if (listenSocket != INVALID_SOCKET) ::closesocket(listenSocket);

    // (3) 关闭所有会话套接字，唤醒各自的接收线程
    {
        auto sessions = Snapshot(false);
        for (const auto& session : sessions) {
            std::lock_guard<std::mutex> lock(session->sendMutex);
            if (session->sock != INVALID_SOCKET) ::shutdown(session->sock, SD_BOTH);
        }
    }

    // (4) 等待所有会话线程收尾
    {
        std::unique_lock<std::mutex> lock(m_activeMutex);
        m_activeCv.wait_for(lock, std::chrono::seconds(8),
                            [this]() { return m_activeSessions == 0; });
    }
    // (5) 继续 join 心跳线程，最后 WSACleanup
}
```

停止服务**先广播系统公告**，让所有客户端知道是服务端主动关闭；**再停止接受新连接；然后逐个唤醒会话线程；最后等待它们全部结束**。第四步用了条件变量加 8 秒上限，是为了防止某个线程因为异常迟迟不退出而把整个程序卡死。

6.3 协议编解码（common/protocol.cpp）

协议模块只做两件事：把“类型 + 字段”的**逻辑帧**编码成字节，或者把字节还原成逻辑帧，所有涉及到长度前缀、转义、字节序的细节都集中在这里，其他模块看不到字节。

```
bool BuildWire(const Frame& f, std::string& wire, std::string* err) {
    std::string payload;
    if (!BuildPayload(f, payload, err)) {
        return false;
    }
    const uint32_t len = static_cast<uint32_t>(payload.size());
    std::string out;
    out.reserve(4 + payload.size());
    out.push_back(static_cast<char>((len >> 24) & 0xFF)); // 大端：高字节在前
    out.push_back(static_cast<char>((len >> 16) & 0xFF));
    out.push_back(static_cast<char>((len >> 8) & 0xFF));
    out.push_back(static_cast<char>(len & 0xFF));
    out += payload;
    wire.swap(out);
    return true;
}
```

编码函数先把字段拼成负载，再在负载前面补 4 个字节的长度。

解码方向要处理两个安全问题：

```
bool ParsePayload(const std::string& payload, Frame& f, std::string* err) {
```

```
    f.type.clear();
    f.fields.clear();
    if (payload.empty()) {
        SetErr(err, "负载为空");
        return false;
    }
    const size_t firstTab = payload.find('\t');
    f.type = (firstTab == std::string::npos) ? payload : payload.substr(0, firstTab);

    size_t pos = firstTab;
    while (pos != std::string::npos) {
        const size_t next = payload.find('\t', pos + 1);
        const std::string raw = (next == std::string::npos)
                                ? payload.substr(pos + 1)
                                : payload.substr(pos + 1, next - pos - 1);
        std::string value;
        if (!UnescapeField(raw, value)) {    // 非法转义 -> 整帧判为非法
            SetErr(err, "字段转义非法");
            return false;
        }
        f.fields.push_back(std::move(value));
        pos = next;
    }
    return true;
}
```

第一个是**长度上限**：接收方在读到 4 字节长度后，会先判断它是否在 1 MiB 以内且不为 0，超过就直接判为非法帧并断开连接，避免有人发一个“长度 4 GB”的帧头把内存撑爆。第二个是**严格的转义校验**：字段里出现无法识别的转义序列时，整帧被判为非法而不是“尽力解析”。

6.4 网络原语（common/net.cpp）

网络层先检查返回值、把错误码翻译成可读文字、把可能只完成一部分的调用补成循环，它没有把套接字包装成对象，所有函数都直接接收 SOCKET 句柄。

发送方向的难点在于 send 不保证一次发完：

```
bool SendAll(SOCKET s, const char* data, size_t len, std::string* err) {
    g_lastError = 0;
    size_t sent = 0;
    while (sent < len) {
        const int n = ::send(s, data + sent, static_cast<int>(len - sent), 0);
        if (n == SOCKET_ERROR) {
            g_lastError = WSAGetLastError();
            if (err != nullptr) *err = WsaErr(g_lastError, "send 失败");
            return false;
        }
        if (n == 0) {
            if (err != nullptr) *err = "send 返回 0，连接已失效";
            return false;
        }
        sent += static_cast<size_t>(n);
    }
    return true;
}
```

循环里要区分返回 0 的情况：send 返回 0 表示连接已失效，必须立即返回失败，否则会变成死循环。

接收方向对应地要“读满一条完整的帧”：

```
bool RecvExact(SOCKET s, char* buf, size_t len, std::string* err) {
    g_lastError = 0;
    size_t got = 0;
    while (got < len) {
        const int n = ::recv(s, buf + got, static_cast<int>(len - got), 0);
        if (n == SOCKET_ERROR) {
            g_lastError = WSAGetLastError();
            if (err != nullptr) *err = WsaErr(g_lastError, "recv 失败");
            return false;
        }
        if (n == 0) {
            if (err != nullptr) *err = (got == 0) ? "对端已关闭连接" : "连接在传输中途被关闭";
            return false;
        }
        got += static_cast<size_t>(n);
    }
    return true;
}
```

RecvFrame 在它之上实现：先读 4 字节帧头得到长度 L，校验 L 合法后再调用 RecvExact 读满 L 字节负载。两个循环叠加，就彻底解决了粘包与半包。

连接方向如果直接调用阻塞的 connect，当对端不可达时界面会卡住几十秒，因此客户端采用“非阻塞连接加 select 等待”的方式，最多等待 5 秒：

```
u_long nonBlocking = 1;
::ioctlsocket(s, FIONBIO, &nonBlocking);
const int rc = ::connect(s, result->ai_addr, static_cast<int>(result->ai_addrlen));
if (rc == SOCKET_ERROR && WSAGetLastError() == WSAEWOULDBLOCK) {
    fd_set writeSet, exceptSet;
    FD_ZERO(&writeSet); FD_ZERO(&exceptSet);
    FD_SET(s, &writeSet); FD_SET(s, &exceptSet);
    ::select(0, nullptr, &writeSet, &exceptSet, &tv); // 最多 5 秒
}
```

连接被拒绝（对方端口没开）时，select 只会在 **except 集合**里置位，这样连接被拒绝可以立刻返回，用户马上就能看到“请先打开 Server.exe”的提示。

6.5 丢包自测工具（tools/proto_test.cpp）

我们设计一个**可量化的手段**来证明它没丢东西，我们单独写了一个命令行客户端 ChatTest.exe：它像普通客户端一样登录，然后连续发送 N 条带序号的消息，再把收到的回显与自己发出的条数逐一比对。

判定逻辑是：统计发送条数、收到自身回显的条数，以及全局序号是否出现跳跃；只有“缺失为 0 且跳跃为 0”才输出 result=PASS 并返回退出码 0，否则打印出具体是哪一条丢失。最后一行固定输出 SUMMARY sent=... echoes=... missing=... frames=... gaps=... result=...。

它是独立进程，走的是与真实客户端完全相同的协议路径，因此它跑通就说明协议、广播、序号三条链路都是通的；而在压力下（200 条快速发送）仍然缺失为 0，说明服务端的广播串行化与客户端的接收循环没有丢事件。

7 程序界面展示及运行说明

7.1 启动服务端

双击 Server.exe 后，程序自动监听 8888 端口并在日志中记下启动时间。界面左侧为在线用户列表，右侧为带时间戳的运行日志，底部状态栏显示监听地址与在线人数。

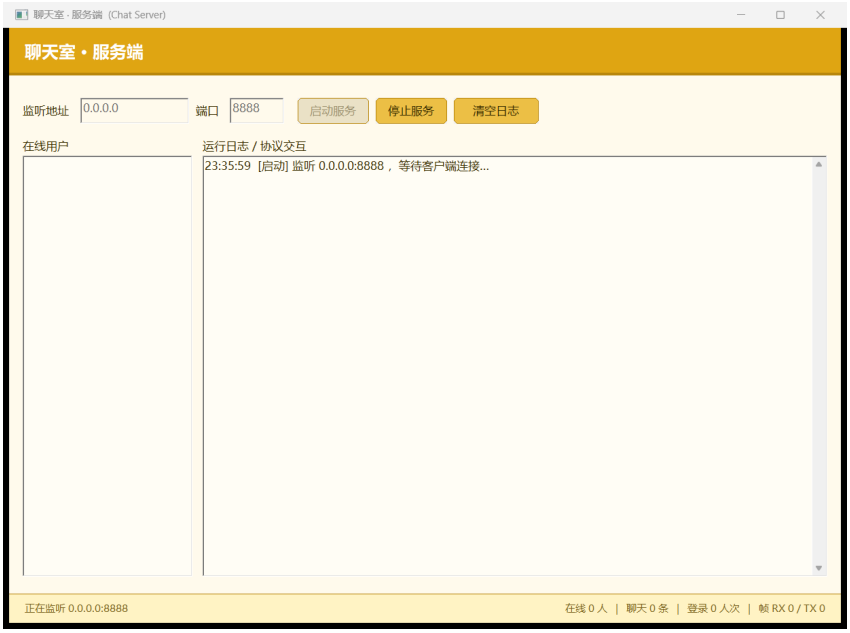


图 7.5: 服务端启动界面

7.2 客户端未连接状态

打开客户端，服务器地址、端口、昵称都已填好，此时“断开”按钮为灰色、“连接”按钮高亮。

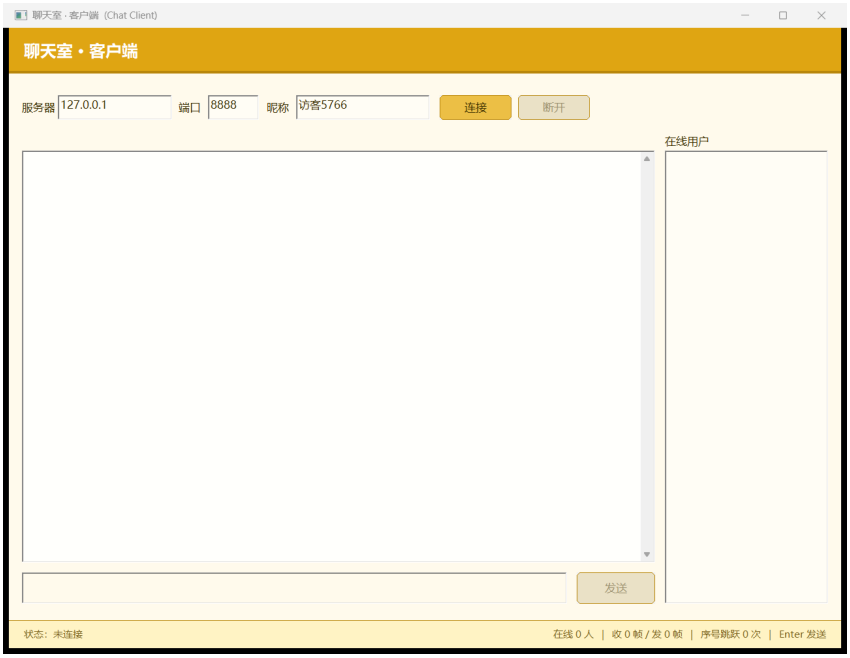


图 7.6: 客户端未连接状态

7.3 客户端连接成功

点击“连接”后，聊天区出现“已连接服务器”与“张三加入了聊天室，当前在线 1 人”两行提示，右侧在线列表中出现了自己。

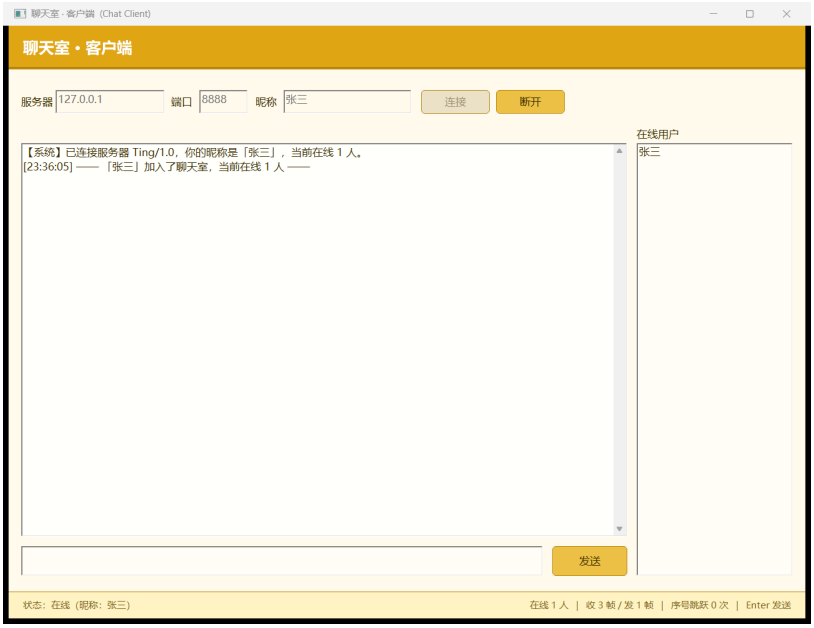


图 7.7: 客户端连接成功

这两行提示分别对应 WELCOME 与 JOIN 两条消息，说明登录握手与成员广播都正常。

7.4 多人聊天

再打开一个客户端，昵称改为“李四”并连接，两个客户端互相发送消息，两边都能实时看到对方的内容。张三的界面如下，他自己发送了一条中英混排的消息“你好，hello”：

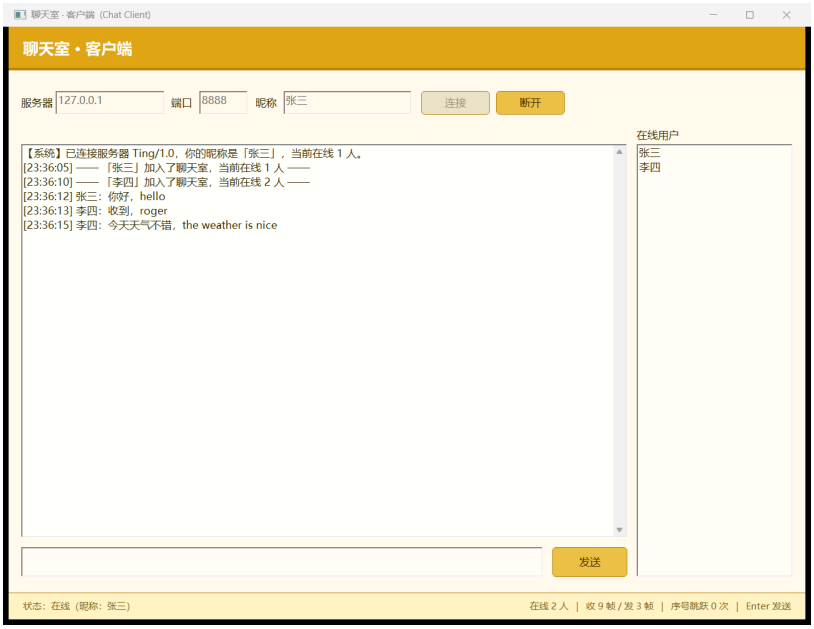


图 7.8: 张三的聊天界面

李四的界面如下，他先回复了“收到，roger”，又发了一条“今天天气不错，the weather is nice”：

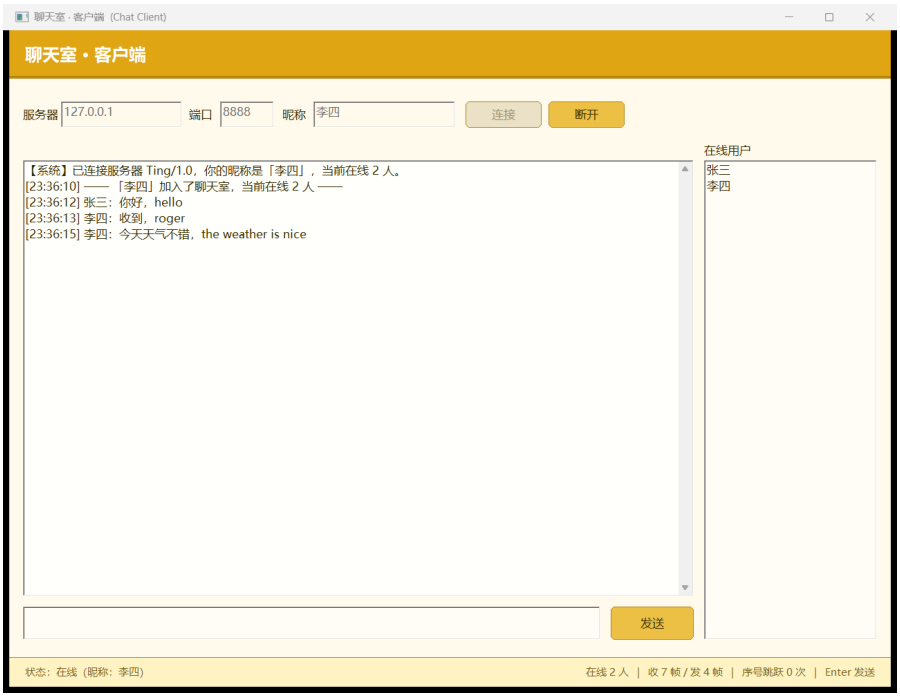


图 7.9: 李四的聊天界面

对比两张截图可以看到**每行开头的 [时: 分: 秒] 完全一致；中文、英文、标点混排显示正常；右侧在线用户列表同步显示两人**，底部状态栏显示“收 N 帧 / 发 M 帧”以及“序号跳跃 0 次”，说明在此之前没有发生任何漏消息。

服务端记录了完整的交互过程：



图 7.10: 服务端日志记录

日志中每条记录都带时间戳，可以看到“连接 → 登录 → 聊天”的完整过程。聊天记录里除了昵称和正文，还带有发送者的**本地序号**，它表示“这是该客户端发出的第几条消息”，用于排查时对账，而广播给所有人的顺序由服务端的全局序号保证。

7.5 退出聊天室

点击“断开”，张三这一侧的状态变化如下：

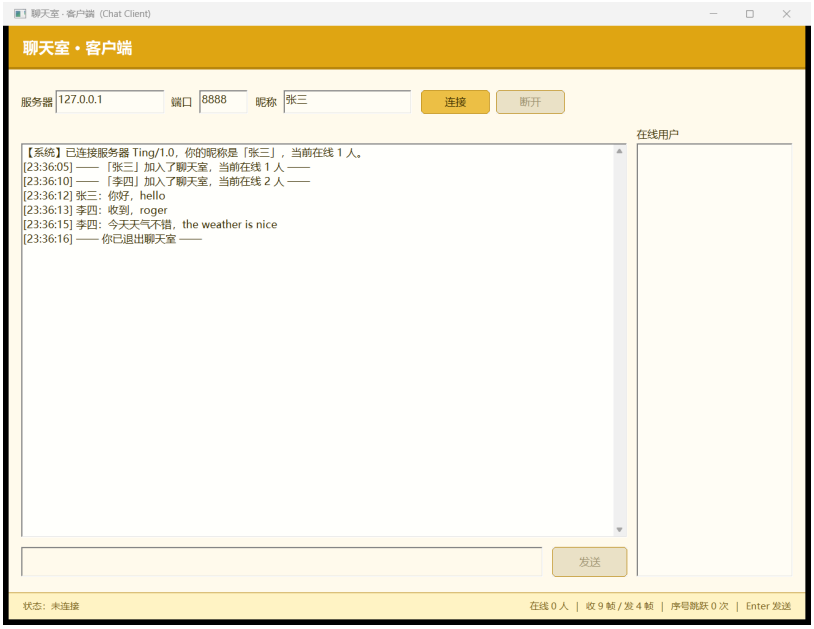


图 7.11: 张三退出

聊天区最后多出一行“——你已退出聊天室——”，状态栏由“在线”变回“未连接”，在线列表清空，“连接”重新可用。

同时，李四那边立刻收到了成员变动通知：

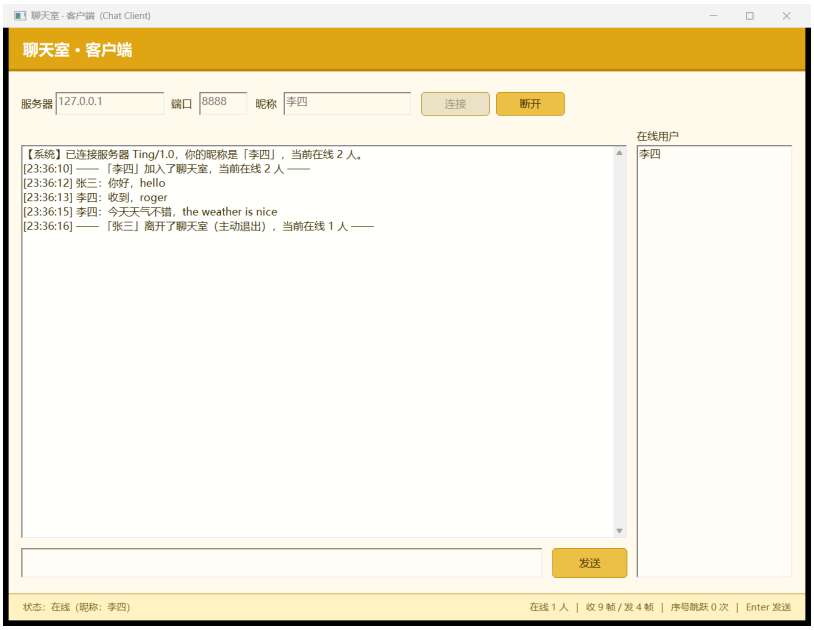


图 7.12: 李四看到张三离开

显示“张三离开了聊天室（主动退出），当前在线 1 人”，括号里的**主动退出**是因为张三在关闭连接之前先发送了 BYE，服务端据此知道这是一次正常退出。

服务端的在线列表与日志也同步更新：



图 7.13: 服务端退出后

在线用户只剩李四，日志中记录了张三的离开及原因。这样，登录、聊天、退出三条主要流程全部验证完毕。

7.6 丢包观察

7.6.1 应用层的自测工具

在服务端保持运行的情况下，用自测工具连续发送 200 条带序号的消息：

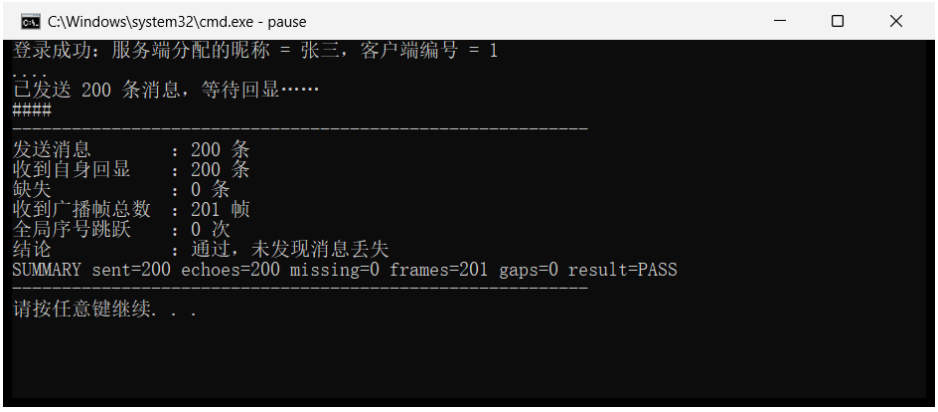


图 7.14: ChatTest 运行结果

可以看到发送 200 条，收到自身回显 200 条，**缺失 0 条**；收到广播帧总数为 201 帧，比 200 条回显多出 1 帧，是因为自己登录时的 JOIN 事件同样带序号、也被计入统计；**全局序号跳跃 0 次**，说明每一条广播都被按顺序收到。

这个结果说明：在 200 条消息连续快速发送的压力下，服务端的广播串行化、客户端的接收循环与分帧逻辑都没有丢失或乱序任何一条消息。

7.6.2 抓包验证

为了从网络层进一步确认，我在回环网卡上抓取了两份数据：chat-conversation.pcapng 记录了一次完整的登录、聊天与退出过程，共 60 个分组；chat-traffic.pcapng 记录了 200 条消息的批量收发过程，共 662 个分组。用 Wireshark 打开前者，可以看到所有通信都发生在 127.0.0.1 的回环接口上，端口为 8888：

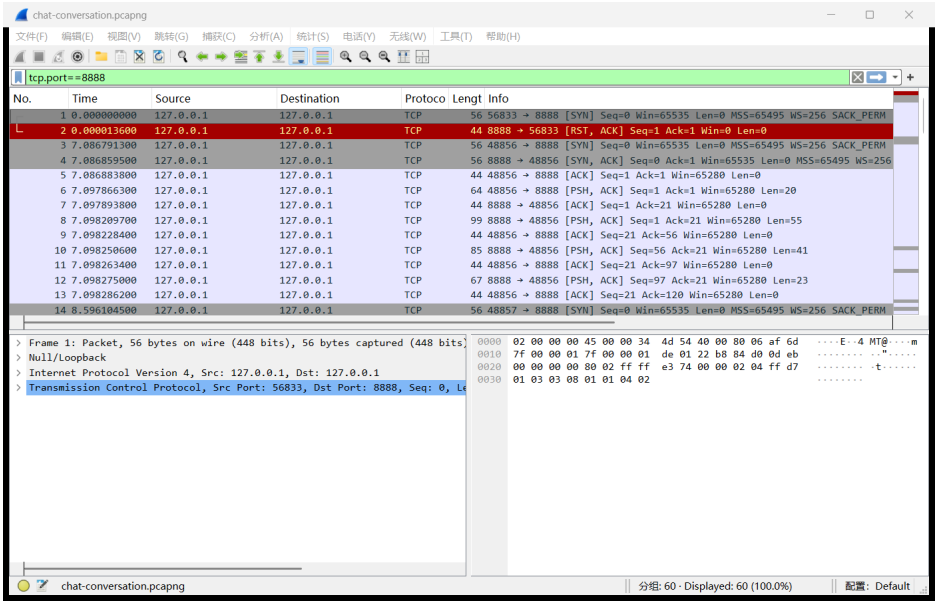


图 7.15: Wireshark 抓包主界面

在显示过滤器中输入 tcp.len > 30，只保留携带应用数据的报文，选中其中一条后，下方十六进制窗口可以直接读出协议内容：

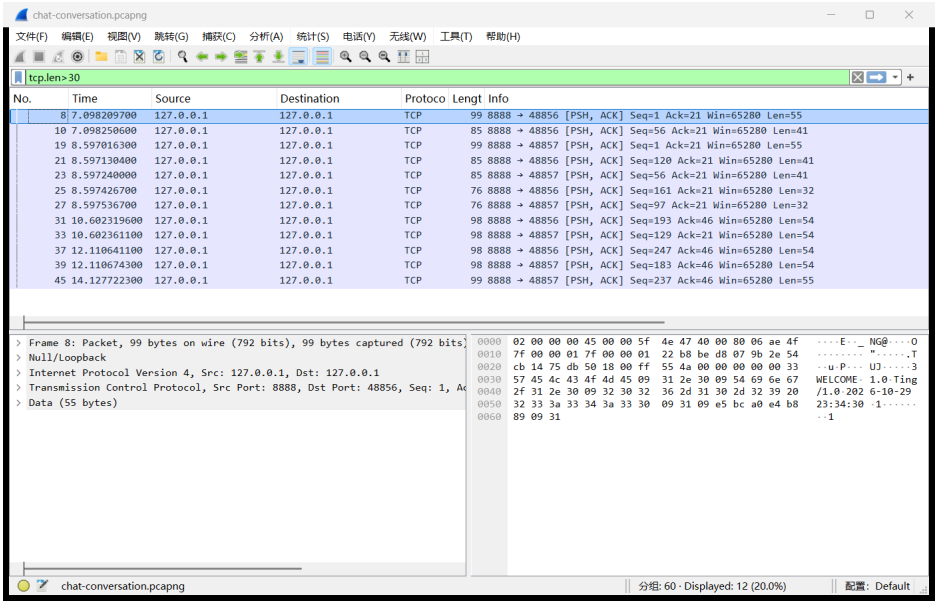


图 7.16: 在 Wireshark 中查看协议内容

十六进制区右侧的 ASCII 部分能清晰看到 WELCOME 1.0 Ting/1.0 2026-10-29... 这样的文本，以及后面的制表符和中文昵称的 UTF-8 字节，这说明**协议在网络上就是可读的文本加长度前缀**，前 4 个字节 00 00 00 33 正是该帧的负载长度（十进制 51），与 4.2 节描述的帧格式完全吻合。

对两份抓包文件分别统计，结果如表 7.2 所示。

统计项	**c**hat-conversation.pcapng	**c**hat-traffic.pcapng
分组总数	60	662
携带应用数据的报文	22	405
TCP 重传	0	0
乱序	0	32
丢失的报文段（lost segment）	0	7
重复 ACK	0	2

表 7.2: 两份抓包文件的统计对比

两份抓包的**重传数都是 0**，说明没有任何一个报文需要重发，也就意味着没有发生真正的丢包。批量抓包中出现的 32 次“乱序”、7 次“lost segment”与 2 次“重复 ACK”不是网络真的丢了包，而是回环网卡抓包时的顺序假象，Wireshark 因此把它标记为乱序；而“lost segment”与“重复 ACK”正是乱序的连带产物。判断是否真的丢包的可靠依据是重传数，而它始终为 0。

把 200 条消息的抓包按 TCP 序号重组后，可以数出 **400 个 CHAT 帧**（200 条上行、200 条下行），与自测工具统计的 echoes=200 完全对应。应用层计数、抓包重组、重传统计三者互相印证，可以确认：**本次实验的消息没有丢失，也没有乱序**。

7.7 运行说明

直接运行 code/exe 下的可执行文件即可。

先双击 Server.exe 启动服务端，程序会自动监听 0.0.0.0:8888，状态栏显示“正在监听”后即可连接；随后双击 Client.exe 启动客户端（可以重复打开多个），确认服务器地址为 127.0.0.1、端口为 8888，填写昵称后点击“连接”即可加入聊天室。

丢包自测在命令行中运行：code\exe\ChatTest.exe –name 测试 –count 200，其中 –count 指定发送条数，–name 指定登录昵称。

8 实验过程中遇到的问题及分析

1、消息粘包与半包，导致聊天记录错乱。

程序最初跑通时，单独发一条消息一切正常，但连续快速发送两条时，对方偶尔会把两条并成一条显示，或者只显示半句话，原因是当发送端连发两条短消息时，它们可能被合并进同一个 TCP 段；接收端缓冲区里先到了半条消息时，recv 也只会返回这半条。

最终的解决方案是在协议中引入**长度前缀**：每条消息前面加 4 个字节表示负载长度，接收方先精确读满 4 字节得到长度 L，再精确读满 L 字节，才得到一个完整的帧，配合 net.cpp 中 send 与 recv 的两个循环，粘包与半包同时被解决。这个过程让我真正理解了 TCP 是字节流，可靠传输解决的是字节会不会丢，而字节如何切分成消息则是应用层的责任。

2、多人同时发言时，不同客户端看到的顺序不一致。

两个客户端同时发送消息时，偶尔会出现“张三那边李四的消息在前，李四那边张三的消息在前”的情况，排查后发现，服务端每个会话线程各自处理消息，两个线程几乎同时执行“取序号”和“逐个人发送”这两步，于是虽然序号本身是递增的，但发送的先后却可能交错。

解决方案是把“取序号 + 编码 + 广播”整体放进同一把**广播锁**里，使广播严格串行：谁先拿到锁，谁就完整地发完一轮，之后才轮到下一个人，同时给每条广播加上全局序号，客户端据此校验连续性。改动之后，无论多少人同时发言，所有客户端收到的顺序都完全一致。

3、服务端重复启动时端口被重复绑定。

调试时曾出现“两个服务端同时运行”的情况：先启动的一个在监听，再次双击 Server.exe 却没有任何报错，两个进程都显示“正在监听 8888”。原因是 Windows 的 SO_REUSEADDR 语义允许在特定条件下重复绑定同一端口，于是第二个进程悄悄抢走了一部分连接，表现为客户端时好时坏。

解决方案是在启动前**主动探测端口**：先尝试用客户端身份连接 127.0.0.1: 端口，如果能连上，说明已经有服务端在跑，直接给出提示并拒绝启动；连不上才真正去 bind 与 listen。这样既避免了重复监听，也让用户得到了明确的错误提示。

9 心得体会

“TCP 可靠”不等于“应用层不用管边界”。做完这次实验，对我影响最大的是 TCP 保证的是字节可靠、有序地到达，但它不告诉你哪里是一条消息的结束，实际写起来才发现，粘包、半包、截断这些问题全都出在应用层。后来我添加了一个 4 字节长度前缀，这让我理解了每一层只解决自己该解决的问题。

多线程的难点是“共享状态”。开一个线程只需要一行代码，难的是想清楚：哪些数据会被多个线程同时访问？该用哪把锁？加锁的顺序会不会死锁？界面为什么不能在网络线程里更新？把“在线名单、广播、每连接发送”拆成三把职责不同的锁之后，程序才真正稳定下来。

日志和抓包很有用。调试期间，服务端日志和 Wireshark 帮我定位了大部分问题，学会用证据说话，比反复改代码试错高效太多；而“重传为 0、乱序是回环抓包假象”这样的判断，也只有在真正看过抓包之后才敢下。

附录 源代码目录结构

本程序采用分层架构组织代码，自下而上分为网络层、协议层、业务层与界面层，共计 19 个源文件，完整目录结构如下：

```
source_code/
+- common/
|  +- platform.h          统一包含 WinSock2.h + windows.h，保证头文件顺序
|  +- net.h / net.cpp     网络层：Winsock 基本函数薄封装
|  +- protocol.h / protocol.cpp 协议层：Ting/1.0 编解码
|  +- text.h / text.cpp   文本工具：UTF-8/UTF-16 转换、时间格式化
|  +- logger.h / logger.cpp 日志文件：线程安全 FileLogger
|
+- client/
|  +- main.cpp            客户端入口：Win32 图形界面窗口
|  +- client.h / client.cpp 客户端核心业务：ChatClient 类
|
+- server/
|  +- main.cpp            服务端入口：Win32 图形界面窗口
|  +- server.h / server.cpp 服务端核心业务：ChatServer 类
|
+- ui/
|  +- theme.h / theme.cpp 界面公共组件：配色、DPI、控件创建、绘制
|  +- ui_channel.h        工作线程→界面线程事件通道
|
+- tools/
   +- proto_test.cpp      丢包自测工具（编译为 ChatTest.exe）
```